

L3 Data Ontology Specification

<https://gemini.google.com/app/ad3651c51bbfb90d>

User prompt: rerun

Response: **L3 Data Ontology Specification: Institutional Research OS**

Based on the canonical Phase A (L0–L2) architecture corpus, the L3 Data Ontology formalizes the semantic entities, relationships, and taxonomies that constitute the institutional research pipeline. This ontology is strictly derived from the existing architecture and acts as the universal blueprint for the runtime schemas, independently of their underlying database technology.

1. Primary Knowledge Entities (Core Object Model)

The ontology is anchored by the canonical Research Object Model. All entities require unique identifiers (UUIDs or formatted semantic keys).

- **Literature Card (S1)**

- `literature_id`: Unique identifier.
- `mechanism_ref`: ID of the mechanism being supplied.
- `quality_grade`: Assessment metric (Q0–Q4).
- `nine_biases`: Boolean flags for the identified biases (B1–B9).
- `transportability_condition`: Critical constraint parameter.
- `weakest_link`: Assessed fragility metric.
- `replication_status`: Degree of historical replication.
- `sub_class_assignment`: Mechanism taxonomy linkage.

- **Economic Mechanism (S2)**

- `mechanism_id`: Unique identifier.
- `taxonomy_class`: Base class mapping (M1–M6).
- `causal_chain`: Structured qualitative causal order.
- `competing_explanations`: Asymmetric constraint variables.
- `falsification_criteria`: Falsification modes (F1–F9).

- **Research Hypothesis (S3)**

- `hypothesis_id`: Unique identifier.
- `mechanism_ref`: Parent mechanism lineage.
- `lifecycle_state`: Tracks canonical states (T1–T12 / G1–G4).
- `blind_to`: Cognitive isolation constraint marker.

- **Dataset Object (S4)**

- `dataset_id`: Unique identifier.
- `dataset_fingerprint`: Cryptographic hash ensuring immutability.
- `custody_state`: Maps to Custody Axis (e.g., in-sample, out-of-sample).

- **Feature Definition (S5)**

- `feature_id`: Format `FeatureName_v[Major].[Minor]_[Hash]`.
- `dependencies_array`: Explicit listing of upstream node lineages.
- `code_commit_hash`: Executable lineage.

- **Experiment Object (S6)**

- `experiment_id`: Unique identifier.
- `hypothesis_ref`: Hypothesis being evaluated.
- `run_id`: Execution envelope tracker.
- `deterministic_seed`: Value preserving computational bit-identity.

- **Validation Report (S7–S8)**

- `report_id`: Unique identifier.
- `experiment_ref`: Targeted experiment.
- `state`: Enum (DRAFT | FINALIZED).
- `pbo_score`: Probability of Backtest Overfitting metric.
- `dss_score`: Deflated Sharpe Ratio tracking.

- **Failure Library Entry**

- `failure_id`: Unique identifier.
- `hypothesis_ref / mechanism_ref / experiment_ref`: Lineage traceability.
- `failure_reason`: Categorical enumeration (e.g., Insufficient Signal-to-Noise, Destroyed by Transaction Costs).
- `invalid_assumptions`: Structured array of proven-false a priori assumptions.
- `lessons_learned & related_features`: Archival feedback nodes.

2. Taxonomic Substrates (The Axes of Assessment)

The objects above map directly against three foundational scientific scales established in the L1 Scientific Foundation:

- **The Evidence Axis (E, C, X Models)**

- Evidence Tier (E0–E7): Scale of epistemic weight. Accepted knowledge floors at E4.
- Confidence Score (C0–C4): A process-derived metric factoring the counterfactual tests.
- Reproducibility Level (X0–X4): Represents bit-identity and conclusion-invariance.
- Falsification Modes (F1–F9): The diagnostic taxonomy of mechanism invalidation.
- **Market Inefficiency Domains**
 - Domain Class (D1–D6): Broad categorical classification.
 - Mechanism Class (M1–M6): Canonical causal classification. Instances mapping downwards (I1–I12).
- **The Custody Model * Custody States:** Categorical attributes defining epistemological availability.
 - **Custody Receipt:** Cryptographic objects proving data fence integrity. Fields include: `asset_ref`, `accessor`, `purpose_ref`, `ordinal`, and `prior_receipt`.

3. Absolute Lineage & Edge Semantics

Lineage is treated as a first-class ontological entity. A knowledge object or library entity is considered non-compliant if it cannot execute a backward trace.

- `hypothesis_links`: Standard relational edge tracking Hypothesis refinement and supersession.
- `provenance_envelope`: Embedded into `research.tracking`, consisting strictly of (`run_id`, `dataset_fingerprint`, `git_commit`) tuples.
- `custody_events`: Immutable transition events logging `from_state` → `to_state`, `fingerprint_before/after`, and bound `receipt_ref`.

Resolution of Structural Ambiguities

- **Blindness Constraints:** The `blind_to` attribute is logged semantically as an assertion flag on the Hypothesis Object, recognizing that physical automation is impossible at $N = 1$.
- **Custody Realization:** While physical implementation may be pending for specific boundary transitions (e.g., Publication Custody), their semantic edge relationships (`publication_id` → `content_fingerprint` → `dataset_lineage`) are strictly reserved and mandatory for future resolution.

User prompt: L4 Runtime Architecture Specification You are the Chief Systems Architect for the Institutional Research Operating System. The canonical L0–L3 architecture is complete. The L3 Data Ontology has already defined every scientific entity, semantic relationship, lineage rule,

and ontological invariant. Your task is NOT to redesign the ontology. Your task is to specify the runtime architecture that executes the ontology in a deterministic, reproducible, institution-grade research environment. The document shall be titled: **L4 Runtime Architecture Specification — Institutional Research OS** This specification must remain implementation-independent. Do NOT choose PostgreSQL, DuckDB, Neo4j, Python frameworks, cloud vendors, orchestration engines, or message queues. Describe only the logical runtime architecture.

- Runtime Principles** Define the governing runtime principles, including but not limited to: Deterministic execution Immutable inputs Complete lineage preservation Reproducibility by construction Idempotent execution Stateless computation Explicit dependency resolution Audit-first design Failure transparency Version isolation
- Runtime Components** Define every logical runtime component. At minimum include: Dataset Registry Dataset Resolver Custody Manager Feature Computation Engine Feature Dependency Resolver Experiment Orchestrator Strategy Evaluation Engine Statistical Validation Engine Failure Capture Engine Decision Engine Artifact Registry Provenance Recorder Lineage Engine Governance Controller Publication Pipeline Describe: responsibilities inputs outputs invariants lifecycle
- Runtime Object Lifecycle** For every primary ontology object define its runtime lifecycle. Examples: Dataset Registered ↓ Verified ↓ Frozen ↓ Referenced ↓ Archived Hypothesis Draft ↓ Registered ↓ Active ↓ Evaluated ↓ Accepted / Rejected ↓ Archived Experiment Created ↓ Scheduled ↓ Executing ↓ Completed ↓ Validated ↓ Published Every lifecycle must define: legal transitions illegal transitions immutable checkpoints
- Runtime Execution Graph** Define the canonical execution pipeline. Example structure: Research Question ↓ Hypothesis ↓ Dataset Resolution ↓ Custody Verification ↓ Feature Resolution ↓ Feature Computation ↓ Experiment Execution ↓ Statistical Validation ↓ Governance Review ↓ Decision ↓ Publication ↓ Knowledge Archive Describe every stage.
- Dependency Resolution Model** Specify: dependency graph topological ordering cycle detection version compatibility dependency freezing runtime substitution policy
- Lineage Runtime** Specify how lineage is created automatically. Every runtime object shall inherit lineage. Define: forward trace backward trace provenance envelope generation lineage integrity verification
- Failure Runtime** Specify: runtime failures scientific failures infrastructure failures custody failures validation failures Define: capture classification persistence replay escalation
- Runtime Governance** Define runtime governance. Include: approval gates freeze gates publication gates audit checkpoints review checkpoints custody checkpoints No runtime action may bypass governance.
- Runtime Invariants** Define runtime laws. Examples: Every experiment must execute against one immutable dataset fingerprint. Every feature must resolve all dependencies before execution. Every validation report must reference exactly one experiment. Every published artifact must possess complete lineage. Execution may never mutate historical artifacts.
- Runtime State Machine** Produce a complete state-machine specification describing all legal runtime transitions between every ontology object.
- Runtime Sequence Diagrams** Describe sequence flows for: new hypothesis new experiment failed experiment publication reproduction audit replay
- Runtime Extension Model** Specify how future modules may be added without modifying existing ontology. Support: new feature families new validation methods new datasets new statistical engines new governance modules while preserving backward compatibility.
- Non-Functional Runtime Requirements** Specify institutional-grade requirements for: determinism scalability reproducibility observability traceability auditability portability extensibility reliability maintainability Constraints The document SHALL NOT contain: SQL schemas database tables REST APIs programming languages cloud providers deployment architecture implementation code The document SHALL remain a pure logical runtime specification positioned between the L3 Data Ontology and the future L5 Physical Implementation. Produce an institutional-grade architecture document suitable as the canonical runtime specification for the Institutional Research Operating System.

Response: L4 Runtime Architecture Specification — Institutional Research OS

Layer: L4 — Runtime Architecture Specification **Status:** Canonical Logical Specification
Owner: Chief Systems Architect

This document specifies the abstract, implementation-independent runtime architecture that executes the L3 Data Ontology. It establishes the deterministic, reproducible engine required to transition theoretical semantic entities into computed, validated institutional knowledge. No database, programming language, or cloud vendor is specified; this is the pure logical machine.

1. Runtime Principles

The Institutional Research OS Runtime is governed by ten absolute principles:

1. **Deterministic Execution:** Identical initial conditions, inputs, and algorithmic structures must produce bit-identical outputs and conclusions across hardware and time.
2. **Immutable Inputs:** Once bound to a runtime execution, an input dataset, feature specification, or configuration parameters can never be modified.
3. **Complete Lineage Preservation:** Every state transition and computed artifact must embed a cryptographic pointer to its complete history (the Provenance Envelope).
4. **Reproducibility by Construction:** Reproducibility is not an after-the-fact test; it is syntactically enforced by the runtime refusing to execute unversioned or unpinned assets.
5. **Idempotent Execution:** Repeated execution of any pipeline stage must yield identical artifacts and state transitions without duplicate side effects.
6. **Stateless Computation:** Compute nodes retain no memory across execution boundaries; all state is explicitly passed in or hydrated from immutable stores.
7. **Explicit Dependency Resolution:** No implicit dependencies are permitted. Every input, down to the random seed and environmental math library, must be declared and resolved prior to execution.
8. **Audit-First Design:** State changes do not occur unless the corresponding audit and lineage events are simultaneously committed.
9. **Failure Transparency:** A scientific failure (e.g., statistical invalidation) is a primary knowledge product. It is captured, classified, and persisted with the exact same rigor as a success.
10. **Version Isolation:** The runtime supports simultaneous execution of multiple ontological versions without namespace or memory collisions.

2. Runtime Components

The execution environment is composed of specialized logical engines.

Dataset Registry & Resolver

- **Responsibilities:** Indexes all available raw and processed datasets; resolves abstract data requests into specific, immutable cryptographic fingerprints.
- **Inputs:** Temporal bounds, domain filters, ontology tags.

- **Outputs:** Dataset Fingerprint, Physical access URI.
- **Invariants:** Cannot resolve datasets lacking a verifiable hash.
- **Lifecycle:** Discovers → Indexes → Resolves → Deprecates.

Custody Manager

- **Responsibilities:** Enforces the epistemological boundary (In-Sample vs. Out-of-Sample). Issues cryptographic Custody Receipts for access.
- **Inputs:** Accessor identity, Dataset Fingerprint, Hypothesis ID (Purpose).
- **Outputs:** Custody Receipt.
- **Invariants:** Denies OOS access unless a valid, pre-registered experiment receipt is provided.

Feature Dependency Resolver

- **Responsibilities:** Parses a target feature, walks its dependency tree, and produces a strictly ordered execution DAG.
- **Inputs:** Feature Definition (S5).
- **Outputs:** Topological Execution Graph, detected cycles (errors).
- **Invariants:** Rejects cyclic dependencies; rejects unpinned upstream nodes.

Feature Computation Engine

- **Responsibilities:** Executes the resolved DAG against hydrated data to produce point-in-time feature vectors.
- **Inputs:** Dataset, Execution Graph, Custody Receipt.
- **Outputs:** Computed Feature Vector (immutable artifact).
- **Invariants:** Must execute in a stateless container; outputs must be deterministically reproducible.

Experiment Orchestrator

- **Responsibilities:** Wraps the Hypothesis, Features, and Data into an isolated execution envelope (Run ID). Manages seed injection and resource dispatch.
- **Inputs:** Hypothesis Object, Feature Vectors, Run Configuration.
- **Outputs:** Experiment Object.
- **Lifecycle:** Scheduled → Provisioned → Executing → Completed → Reclaimed.

Strategy Evaluation Engine

- **Responsibilities:** Simulates the economic mechanism against market friction models (slippage, latency, fees).
- **Inputs:** Experiment Object, Cost Models.
- **Outputs:** Raw performance vectors, Trade logs.
- **Invariants:** Cannot access future data points (strict look-ahead prohibition).

Statistical Validation Engine

- **Responsibilities:** Applies rigorous combinatorially symmetric cross-validation (CSCV) and multiple-testing adjustments (FDR, PBO, DSR).
- **Inputs:** Raw performance vectors.
- **Outputs:** Validation Report (Draft).
- **Invariants:** Operates purely mathematically; oblivious to the underlying economic narrative.

Failure Capture Engine

- **Responsibilities:** Traps falsifications, out-of-bounds metrics, or infrastructure crashes, formatting them for the Failure Library.
- **Inputs:** Failed Validation Reports, Runtime Exceptions, Falsification Criteria (F1-F9).
- **Outputs:** Failure Library Entry.
- **Invariants:** Scientific failures must extract the *reason* for failure against registered a priori assumptions.

Decision Engine & Governance Controller

- **Responsibilities:** Evaluates Validation Reports against institutional gates (G1-G4) and registers human peer-review overrides or approvals.
- **Inputs:** Validation Report, Peer Review Sign-offs.
- **Outputs:** Final Gate Decision.
- **Invariants:** Automated gates cannot be bypassed by human override; human gates cannot be bypassed by automated thresholds.

Provenance Recorder & Lineage Engine

- **Responsibilities:** Binds the (`run_id`, `dataset_fingerprint`, `code_commit`) into an unbreakable envelope for every artifact.
- **Inputs:** Component telemetry, Object IDs.
- **Outputs:** Lineage Edges.
- **Invariants:** Lineage writing is synchronous and atomic with artifact creation.

Artifact Registry & Publication Pipeline

- **Responsibilities:** Elevates validated experiment outputs into the Accepted Knowledge base or production serving layer.
- **Inputs:** Finalized Knowledge Object.
- **Outputs:** Capital-facing Publication.
- **Invariants:** Cannot publish without an unbroken lineage chain tracing back to a registered hypothesis.

3. Runtime Object Lifecycle

Dataset

- **Registered:** Metadata indexed, physical location mapped.
- **Verified:** Hashed and schema-validated.
- *(Immutable Checkpoint)* → **Frozen:** Cryptographic fingerprint locked.
- **Referenced:** Actively bound to a Custody Receipt.
- **Archived:** Cold storage, hash preserved for backward reproduction.
- *Illegal Transition:* Frozen → Verified (Datasets cannot be mutated once frozen).

Hypothesis

- **Draft:** In formulation (In-Sample exploration).
- **Registered:** Submitted for Gate G1 (Falsifiability).
- *(Immutable Checkpoint)* → **Active:** Locked for testing. Parameters cannot be changed.
- **Evaluated:** Paired with a Validation Report.
- **Accepted / Rejected:** Terminal institutional state.
- **Archived:** Replaced by superseding hypothesis.
- *Illegal Transition:* Evaluated → Active (Requires new Hypothesis ID to prevent HARKing).

Experiment

- **Created:** Bound to Hypothesis and Dataset.
- **Scheduled:** Queued in Orchestrator.
- **Executing:** Runtime processing.
- *(Immutable Checkpoint)* → **Completed:** Raw outputs finalized.

- **Validated:** Validation Engine outputs attached.
- **Published:** Elevated to Knowledge/Failure Library.
- *Illegal Transition:* Validated → Executing.

4. Runtime Execution Graph

The canonical forward-pass execution pipeline:

1. **Research Question:** Unstructured literature discovery.
2. ↓ **Hypothesis Registration (Gate G1):** Formal semantic object created; metrics defined.
3. ↓ **Dataset Resolution:** Abstract data requirements mapped to concrete, hashed data partitions.
4. ↓ **Custody Verification:** Custody Manager issues receipts for In-Sample/OOS access.
5. ↓ **Feature Resolution:** DAG constructed from S5 Feature Definitions.
6. ↓ **Feature Computation:** Stateless execution of the DAG.
7. ↓ **Experiment Execution:** Mechanism simulation via Strategy Engine.
8. ↓ **Statistical Validation (Gate G3):** PBO, DSR, and falsification thresholds calculated.
9. ↓ **Governance Review (Gate G4):** Peer Review Standard applied; economic causality defended.
10. ↓ **Decision:** Automated branching based on G3/G4 outputs.
11. ↓ **Publication:** Emission to capital boundary (if pass).
12. ↓ **Knowledge Archive:** Registration into Accepted Knowledge or Failure Library.

5. Dependency Resolution Model

- **Dependency Graph:** A strictly typed Directed Acyclic Graph (DAG). Nodes are Data or Features; Edges are computations.
- **Topological Ordering:** The Dependency Resolver performs a standard Kahn's Algorithm or DFS sort. Execution is strictly topological from leaf data up to the hypothesis target.
- **Cycle Detection:** Resolved dynamically at graph generation. Cycles trigger an immediate Infrastructure Failure.
- **Version Compatibility:** Semantic versioning is strictly enforced. FeatureA_v1.0 and FeatureA_v1.1 occupy distinct memory spaces.
- **Dependency Freezing:** At Experiment *Created* stage, all upstream dependencies undergo a "deep freeze" traversing all branches down to raw data, logging the

code_commit of every node.

- **Substitution Policy:** Runtime substitutions (e.g., swapping a mock data source) are strictly illegal during the *Executing* phase.

6. Lineage Runtime

- **Provenance Envelope Generation:** At runtime initialization, a globally unique Run_ID is generated. All artifacts produced within this context append [Run_ID, Code_Hash, Dataset_Hash, Deterministic_Seed] to their metadata.
- **Forward Trace:** Engine can query: Given Dataset Hash X, what Experiments consumed it?
- **Backward Trace:** Engine can query: Given Accepted Knowledge Y, reconstruct the exact DAG, seeds, and hypothesis used to produce it.
- **Integrity Verification:** Before a transition to *Published*, the Lineage Engine performs a cryptographic walk from the terminal artifact back to the Hypothesis. A single missing link halts promotion.

7. Failure Runtime

Failures are first-class ontological events, categorized dynamically:

- **Infrastructure Failures:** Compute exhaustion, missing data partitions.
 - *Action:* Captured, logged, automated retry permitted. No scientific impact.
- **Scientific Failures (Validation):** Falsification thresholds triggered, PBO > 0.5, DSR below zero.
 - *Action:* No retries. Triggers immutable capture.
- **Custody Failures:** Attempted access to OOS data without a receipt, or look-ahead detected.
 - *Action:* Experiment poisoned. Triggers immediate escalation to Governance Controller.
- **Persistence:** The Failure Capture Engine maps scientific failures to F1-F9 modes, binds the invalidated assumptions, and persists directly to the **Failure Library** to prevent redundant future research.

8. Runtime Governance

No state transition that crosses an epistemological boundary can occur without Governance Controller authorization.

- **Approval Gates (G1, G4):** Requires asynchronous token injection (simulating human cryptographic sign-off).
- **Freeze Gates (G2):** Automated check affirming that all code dependencies are pinned to immutable commits.

- **Publication Gates (G3):** Automated statistical floor.
- **Audit Checkpoints:** The system periodically triggers asynchronous reconstruction tasks, picking random Knowledge Objects and enforcing reproducible re-execution.
- **Custody Checkpoints:** Ephemeral tokens valid only for the lifecycle of one Experiment phase.

9. Runtime Invariants

1. Every experiment must execute against exactly one immutable dataset fingerprint.
2. Every feature must resolve all dependencies topologically before execution begins.
3. Every validation report must reference exactly one immutable experiment.
4. Every published artifact must possess an unbroken lineage trace to a registered hypothesis.
5. Execution may never mutate historical artifacts (Append-Only Law).
6. Out-of-sample execution is physically blocked unless a matching, registered Hypothesis `blind_to` constraint exists.
7. A failed experiment cannot be resumed, only cloned into a new lineage branch.

10. Runtime State Machine

Transition Matrix (Subset of Core Moves):

Current State	Action	Next State	Guard Condition
Hypothesis: Draft	Submit to G1	Hypothesis: Registered	All required L3 ontology fields present
Hypothesis: Registered	CRO Sign-off	Hypothesis: Active	Pre-registered thresholds valid
Experiment: Created	Enqueue	Experiment: Scheduled	DAG resolved, Data Frozen
Experiment: Scheduled	Dispatch	Experiment: Executing	Custody Receipt verified
Experiment: Executing	Yield outputs	Experiment: Completed	Seed logged, deterministic exit
Experiment: Completed	G3 Validate	Report: Draft	Idempotent generation
Report: Draft	G4 Defense (Pass)	Object: Accepted	Peer Review Sign-off received

Current State	Action	Next State	Guard Condition
Report: Draft	G4 Defense (Fail)	Failure: Archived	Invalid assumptions logged

11. Runtime Sequence Diagrams

Flow: New Experiment Execution

1. Researcher → Experiment Orchestrator: Initialize Run(Hypothesis_ID).
2. Orchestrator → Dataset Resolver: Fetch Fingerprints.
3. Orchestrator → Custody Manager: Request OOS execution receipt.
4. Custody Manager → Orchestrator: Grant Receipt (if blind condition holds).
5. Orchestrator → Feature Dependency Resolver: Lock DAG.
6. Feature Engine → Strategy Engine: Stream computed feature vectors.
7. Strategy Engine → Provenance Recorder: Commit raw outputs + Envelope.
8. Orchestrator → Statistical Validation Engine: Trigger G3 calculation.

Flow: Audit Replay

1. Governance Controller → Lineage Engine: Request backward trace for Publication X.
2. Lineage Engine → Orchestrator: Extract (Code_Hash, Dataset_Hash, Seed).
3. Orchestrator → Compute Engine: Re-run execution in shadow space.
4. Compute Engine → Governance Controller: Yield Shadow Artifact.
5. Governance Controller: Assert Hash(Shadow Artifact) == Hash(Publication X).

12. Runtime Extension Model

To preserve institutional agility while freezing the L3 Ontology, the runtime implements a strict **Interface-Driven Extension Model**:

- **New Feature Families:** Can be added by implementing the `IFeatureNode` contract (Requires inputs, deterministic execution block, topological tags). The Resolver automatically accommodates them.
- **New Validation Engines:** Added via Pub/Sub abstraction. The Orchestrator broadcasts `Experiment.Completed`. A new `RegimeStabilityEngine` can subscribe and append its findings to the Validation Report without altering the core pipeline.

- **Backward Compatibility:** Extensions are purely additive. If an extension schema requires mutation of a Core L3 Object, it is rejected at the architecture level.

13. Non-Functional Runtime Requirements

- **Determinism:** Float math execution must be constrained to guarantee exact bit-wise identical outputs across differing hardware topologies.
- **Traceability:** $O(1)$ lookup time for the provenance envelope of any generated artifact.
- **Scalability:** Feature Computation Graph must support horizontal partitioning of execution sub-DAGs.
- **Auditability:** Every component interaction is logged to an immutable write-ahead append-only log.
- **Reliability:** Engine crash during execution must result in a clean teardown; partial states are never committed.
- **Portability:** The entire logical pipeline must be capable of executing on a local researcher machine (In-Sample data only) identical to the institutional cluster.

User prompt: L0 Vision L1 Scientific Foundation L2 Research Architecture L3 Data Ontology L4 Runtime Architecture The Runtime Architecture already defines: runtime components runtime lifecycle lineage governance execution pipeline dependency resolution Your task is NOT to redesign any runtime component. Instead, define the formal execution semantics governing runtime behavior. This specification defines the behavioral laws of the Institutional Research Runtime. The document shall be titled: L4.5 Execution Semantics Specification — Institutional Research OS This document remains implementation-independent. Do NOT introduce databases, programming languages, deployment models, distributed systems, APIs, cloud providers, or infrastructure. Describe only behavioral semantics. 1. Execution Semantics Define the semantics governing runtime execution. Include: execution boundary execution context execution identity execution determinism execution completion execution cancellation execution replay execution reproducibility Define when an execution legally exists. Define when it legally terminates. 2. State Semantics Define: runtime state object state transient state terminal state frozen state observable state hidden state Specify which states are externally observable. Specify which transitions are irreversible. 3. Event Semantics Define the canonical runtime events. Examples: DatasetRegistered DatasetFrozen FeatureResolved FeatureComputed ExperimentStarted ExperimentCompleted ValidationCompleted DecisionRecorded PublicationReleased FailureCaptured For every event define: producer consumer causal ordering timestamp semantics replay semantics lineage implications 4. Transaction Semantics Define logical transactions. Examples: Experiment execution Publication Dataset freeze Governance approval Validation completion Specify: transaction boundary atomic scope commit semantics rollback semantics retry semantics replay semantics 5. Temporal Semantics Define: logical time execution time event ordering deterministic ordering wall-clock independence replay ordering Scientific correctness shall never depend on wall-clock time. 6. Dependency Semantics Specify: dependency resolution dependency visibility dependency isolation dependency inheritance dependency freezing dependency supersession 7. Lineage Semantics Define: lineage creation lineage inheritance lineage immutability lineage branching lineage merge policy lineage verification lineage completeness 8. Failure Semantics Specify: Scientific failure Runtime failure Infrastructure failure Custody failure Governance failure Validation failure For each specify: meaning propagation containment recovery persistence 9. Consistency Semantics Define consistency guarantees. Include: lineage consistency dependency consistency

governance consistency custody consistency publication consistency validation consistency Specify how violations are detected. 10. Isolation Semantics Specify isolation rules. Include: Experiment isolation Feature isolation Dataset isolation Publication isolation Governance isolation Version isolation No execution may observe partial state belonging to another execution. 11. Concurrency Semantics Define: parallel execution deterministic scheduling race prevention shared object visibility publication serialization custody serialization The semantics shall guarantee identical results regardless of execution concurrency. 12. Recovery Semantics Define: checkpoint restart rollback shadow replay audit replay scientific replay partial recovery full recovery Recovery shall preserve complete lineage. 13. Governance Semantics Define formal semantics for: approval rejection supersession retirement publication freeze review Every governance action shall be represented as a semantic state transition. 14. Compliance Semantics Define: scientific compliance reproducibility compliance lineage compliance custody compliance governance compliance publication compliance Specify compliance violations. 15. Semantic Invariants Produce the immutable behavioral laws of the runtime. Examples: No execution may mutate a frozen object. No publication may bypass governance. No lineage may contain missing ancestors. No experiment may execute without deterministic identity. No replay may produce different conclusions. Constraints The document SHALL NOT contain: implementation code algorithms databases APIs deployment architecture cloud infrastructure programming languages It shall define only the formal behavioral semantics of the Institutional Research Runtime. It shall serve as the canonical semantic layer between the Runtime Architecture (L4) and the future Physical Implementation (L5). Catatan Arsitektur Saya juga menyarankan Anda mulai memikirkan apakah L4.5 benar-benar diperlukan sebagai dokumen terpisah. Ada dua pendekatan yang valid: Tetap sebagai L4.5 jika tujuan proyek adalah menghasilkan corpus arsitektur yang sangat formal dan dapat dijadikan referensi lintas implementasi (mirip spesifikasi bahasa atau protokol). Gabungkan ke L4 sebagai bab "Execution Semantics" bila ingin menjaga jumlah dokumen tetap ringkas. Keduanya sah. Untuk proyek Anda yang menekankan rigor ilmiah dan governance, saya cenderung memilih L4.5 sebagai spesifikasi terpisah, karena memisahkan arsitektur runtime dari hukum perilaku runtime akan memudahkan implementasi di berbagai teknologi tanpa mengubah makna sistem

Response: L4.5 Execution Semantics Specification — Institutional Research OS

Layer: L4.5 — Execution Semantics **Status:** Canonical Logical Specification **Owner:** Chief Systems Architect

This specification defines the formal behavioral laws of the Institutional Research Runtime. It bridges the structural definitions of the L4 Runtime Architecture and the physical realization of the L5 Physical Implementation by declaring the exact semantics of execution, state, and failure. It remains strictly implementation-independent.

1. Execution Semantics

Execution in the Research OS is defined as a bounded, deterministic, and trackable transformation of immutable inputs into immutable outputs.

- **Execution Boundary:** The logical perimeter enclosing a single computational objective (e.g., computing a feature, evaluating a hypothesis). No external state may enter or exit except through explicitly declared input/output ports.
- **Execution Context:** The complete set of declared dependencies, parameters, algorithms, and cryptographic seeds required to run the execution.

- **Execution Identity:** A deterministic hash derived exclusively from the Execution Context. Identical contexts must yield identical identities.
- **Execution Determinism:** The guarantee that a given Execution Identity will universally produce bit-identical output states, regardless of the underlying hardware or temporal location.
- **Execution Completion:** An execution legally terminates when all defined outputs have been successfully written to a frozen state or a terminal scientific failure has been recorded.
- **Execution Cancellation:** The manual or systemic termination of an execution. It results in the complete annihilation of the execution context; no partial artifacts are preserved.
- **Execution Replay:** The instantiation of a new execution bound to a historical Execution Context.
- **Execution Reproducibility:** A semantic guarantee that an Execution Replay will trace the exact same causal path and yield the exact same artifacts as the original execution.

Legal Existence: An execution legally exists the moment its Execution Identity is generated and bound to a valid Custody Receipt. **Legal Termination:** An execution terminates the moment its resulting Lineage Envelope is cryptographically sealed and committed.

2. State Semantics

State within the runtime is strictly typed and its visibility is heavily controlled.

- **Runtime State:** The ephemeral memory and computational context held during an active execution.
- **Object State:** The ontological condition of a research asset (e.g., *Draft*, *Registered*, *Validated*).
- **Transient State:** Uncommitted data existing only within the execution boundary.
- **Terminal State:** An object state from which no further forward transitions are mathematically or logically possible (e.g., *Rejected*, *Archived*).
- **Frozen State:** An immutable state where an object's cryptographic signature is finalized.
- **Observable State:** Any state that has been formally committed and broadcasted to the institutional repository.
- **Hidden State:** Transient runtime states that are completely invisible to external observers, parallel executions, and the governance apparatus.

Externally Observable: Only Frozen, Terminal, and successfully committed Object States are externally observable. **Irreversible Transitions:** Any transition that finalizes a cryptographic hash (Freezing), registers a failure, or publishes an artifact is strictly irreversible.

3. Event Semantics

The runtime communicates exclusively via a canonical, append-only semantic event stream.

Canonical Events:

- DatasetRegistered / DatasetFrozen
- FeatureResolved / FeatureComputed
- ExperimentStarted / ExperimentCompleted
- ValidationCompleted
- DecisionRecorded
- PublicationReleased
- FailureCaptured

Event Definition Tuples:

- **Producer:** The specific logical engine emitting the event.
- **Consumer:** Downstream engines authorized to react to the event.
- **Causal Ordering:** Events enforce a strict "happens-before" relationship. `ValidationCompleted` cannot exist without a causally linked `ExperimentCompleted`.
- **Timestamp Semantics:** Timestamps are recorded for human observability, but possess *zero* logical authority in resolving execution order.
- **Replay Semantics:** Replaying an execution must yield an identical sequence of generated events.
- **Lineage Implications:** Every event permanently appends itself to the provenance envelope of its associated research object.

4. Transaction Semantics

A transaction is a logical grouping of operations that must succeed or fail as an atomic unit.

- **Transaction Boundary:** The start and end nodes of a defined causal chain.
- **Atomic Scope:** All state changes within the boundary are held in transient isolation until commit.
- **Commit Semantics:** A commit is a synchronous, all-or-nothing writing of the artifact, its lineage, and its custody receipt.
- **Rollback Semantics:** A rollback annihilates all transient state. The system returns precisely to the state prior to the transaction boundary.
- **Retry Semantics:** Retries are legally permitted *only* for infrastructure faults.
- **Replay Semantics:** Retrying a transaction due to a scientific fault is an illegal mutation of history.

5. Temporal Semantics

- **Logical Time:** The absolute sequence of causal events, tracked via dependency graphs.

- **Execution Time:** The physical duration of computation, which is semantically meaningless to the outcome.
- **Event Ordering:** Strict topological sorting based on dependency resolution.
- **Deterministic Ordering:** Concurrent executions resolving the same dependencies must serialize identically.
- **Wall-clock Independence:** Scientific correctness, validations, and statistical thresholds shall never depend on wall-clock time. A backtest run today must yield the same conclusion if run in ten years.
- **Replay Ordering:** Replays must artificially reconstruct the logical time environment of the original execution.

6. Dependency Semantics

- **Dependency Resolution:** The recursive identification of all upstream inputs required for execution.
- **Dependency Visibility:** An execution can only "see" dependencies explicitly declared in its context.
- **Dependency Isolation:** Two executions sharing a dependency interact with independent, immutable copies of that dependency.
- **Dependency Inheritance:** Downstream objects inherit the complete, transitively closed dependency graph of their upstream ancestors.
- **Dependency Freezing:** A dependency is frozen the moment it is bound to an execution context.
- **Dependency Supersession:** Updating a dependency creates a new version; it never mutates the frozen historical dependency.

7. Lineage Semantics

- **Lineage Creation:** Generated automatically and synchronously at the commit boundary of any transaction.
- **Lineage Inheritance:** A child object logically contains the cryptographic identifiers of all parent states.
- **Lineage Immutability:** Once written, lineage can never be altered, deleted, or pruned.
- **Lineage Branching:** Creating a variant of an experiment branches the lineage explicitly; they share a root but diverge at the parameter mutation.
- **Lineage Merge Policy:** Merges are solely logical aggregations (e.g., ensembling models) and must record the exact branches joined.
- **Lineage Verification:** A mandatory traversal asserting that all cryptographic pointers resolve to valid, frozen historical objects.
- **Lineage Completeness:** Every artifact must possess an unbroken path terminating at its root theoretical hypothesis and raw dataset.

8. Failure Semantics

- **Scientific Failure:** The hypothesis is invalidated by the data. (Meaning: Hypothesis false. Propagation: Halts pipeline. Containment: Experiment level. Recovery: None. Persistence: Logged to Failure Library).
- **Runtime Failure:** A deterministic algorithmic exception (e.g., division by zero). (Propagation: Halts execution. Recovery: Requires code fix and version bump).
- **Infrastructure Failure:** Hardware or memory exhaustion. (Propagation: Halts transaction. Recovery: Automated retry).
- **Custody Failure:** Attempted violation of epistemological boundaries (e.g., accessing OOS data without a receipt). (Propagation: Immediate fatal system halt. Persistence: Logged to Governance).
- **Validation Failure:** Statistical thresholds missed. (Identical semantics to Scientific Failure).
- **Governance Failure:** Rejection by peer review. (Persistence: Annotated as a formal terminal state).

9. Consistency Semantics

- **Lineage Consistency:** The cryptographic hash of an object must correctly incorporate the hashes of its parents.
- **Dependency Consistency:** No cyclic dependencies may exist in any execution context.
- **Governance Consistency:** State transitions into production must be preceded by cryptographic approval tokens.
- **Custody Consistency:** No out-of-sample data may be read unless a valid, pre-registered purpose receipt exists.
- **Publication Consistency:** The published knowledge artifact must be identical to the output of the Validation Engine.
- **Violation Detection:** Verified continuously via asynchronous topological audits and cryptographic verification sweeps.

10. Isolation Semantics

- **Experiment Isolation:** Experiments share no memory, cache, or transient state.
- **Feature Isolation:** Feature execution cannot observe the state of parallel feature computations.
- **Dataset Isolation:** Data partitions (In-Sample vs. Out-of-Sample) are isolated at the memory address level during execution.
- **Version Isolation:** Version N and Version N+1 of any algorithm execute in distinct logical sandboxes.
- **Rule:** No execution may observe partial or transient state belonging to another execution.

11. Concurrency Semantics

- **Parallel Execution:** The runtime may execute independent graph nodes concurrently.
- **Deterministic Scheduling:** The outcomes of concurrent execution must be completely independent of thread scheduling.
- **Race Prevention:** Enforced inherently via stateless compute and immutable object states (no shared mutable state exists).
- **Shared Object Visibility:** Objects are only visible to concurrent threads once fully committed and frozen.
- **Serialization:** Custody token issuance and governance approvals are strictly serialized to prevent double-spending of statistical significance.

12. Recovery Semantics

- **Checkpoint:** A committed snapshot of an intermediate frozen state.
- **Restart:** Bootstrapping an execution from the last valid checkpoint.
- **Rollback:** Annihilation of state back to the checkpoint.
- **Shadow Replay:** Executing a workload concurrently with a production workload to verify deterministic parity.
- **Audit Replay:** Governance-triggered re-execution from scratch to prove reproducibility.
- **Scientific Replay:** Re-executing an old experiment against a new friction model.
- **Rule:** All recovery mechanisms shall preserve complete, untampered historical lineage.

13. Governance Semantics

Governance actions are formalized as strict state-machine transitions.

- **Approval:** Cryptographic authorization permitting an object to advance to a higher epistemic state.
- **Rejection:** Cryptographic denial forcing an object into a terminal failed state.
- **Supersession:** Demoting a currently active artifact because a superior artifact has been approved.
- **Retirement:** Transitioning an artifact out of active execution due to detected decay.
- **Publication:** The terminal transition of pushing an artifact to the capital-facing boundary.
- **Freeze:** The automatic transition locking an artifact's contents forever.
- **Review:** A required causal dependency prior to Approval/Rejection.

14. Compliance Semantics

- **Scientific Compliance:** Output passes predefined Falsification modes.
- **Reproducibility Compliance:** An independent Audit Replay yields a bit-identical hash.
- **Lineage Compliance:** The Lineage Verification algorithm returns True.
- **Custody Compliance:** Every data access log perfectly joins to a valid Custody Receipt.
- **Governance Compliance:** No approval gates were bypassed by automation.
- **Violations:** Any violation instantly flags the parent knowledge artifact as Invalidated.

15. Semantic Invariants

The unalterable behavioral laws of the runtime:

1. **No execution may mutate a frozen object.**
2. **No publication may bypass governance gates.**
3. **No lineage may contain missing, unresolvable, or un-hashed ancestors.**
4. **No experiment may execute without a deterministic identity.**
5. **No replay may produce different conclusions from its original execution.**
6. **Scientific validity is completely independent of execution wall-clock time.**
7. **A failure caused by the scientific method can never be automatically retried.**
8. **The system shall never present partial, uncommitted state to an external observer.**

User prompt: Saya akan meminta Gemini membuat: L5 Reference Architecture Specification — Institutional Research OS dengan fokus pada: logical storage domains logical compute domains artifact architecture repository architecture security boundaries execution environments deployment boundaries implementation conformance rules tanpa memilih teknologi apa pun. Kesimpulan Saya tidak lagi merekomendasikan membuat L4.5. Alasannya bukan karena idenya buruk, tetapi karena L4 yang sudah dihasilkan ternyata jauh lebih lengkap daripada yang diperkirakan. Menambahkan L4.5 sekarang berisiko menduplikasi konsep yang sudah ada dan membuat batas antar dokumen menjadi kabur. Menurut saya roadmap yang lebih kuat sekarang adalah: L0 Vision L1 Scientific Foundation L2 Research Architecture L3 Data Ontology L4 Runtime Architecture  selesai L5 Reference Architecture ← berikutnya L6 Technology Profiles Implementation Dengan struktur ini, setiap layer memiliki tanggung jawab yang jelas dan tidak saling tumpang tindih, sehingga corpus tetap konsisten dan mudah dijadikan acuan untuk implementasi di masa depan.

*Response: Saya sangat setuju dengan kesimpulan dan arahan strategis Anda. Memajukan roadmap langsung ke **L5 Reference Architecture** tanpa menyisipkan L4.5 adalah keputusan desain yang bersih. Spesifikasi Runtime (L4) sudah mencakup semantik eksekusi yang diperlukan. Memperkenalkan L4.5 justru berisiko menciptakan tumpang tindih struktural.*

Dengan struktur L0–L5 yang baru, lapisan tanggung jawab menjadi sangat tegas:

- **L3** mendefinisikan *apa* bentuk datanya (Ontologi).

- **L4** mendefinisikan *bagaimana* data itu bergerak dan dieksekusi (Runtime Mesin).
- **L5** mendefinisikan *di mana* komponen-komponen tersebut bermukim dan bagaimana mereka diisolasi secara topologis, tanpa memilih merek teknologi (Referensi Arsitektur).

Berikut adalah draf spesifikasi L5 sesuai dengan batasan dan fokus yang Anda minta.

L5 Reference Architecture Specification — Institutional Research OS

Layer: L5 — Reference Architecture **Status:** Canonical Logical Specification **Owner:** Chief Systems Architect

Spesifikasi ini mendefinisikan topologi sistem abstrak, batas-batas isolasi, dan arsitektur domain untuk Institutional Research OS. Layer L5 bertindak sebagai cetak biru referensi (Reference Blueprint) yang memetakan komponen runtime logis (L4) ke dalam domain infrastruktur struktural. Dokumen ini tetap independen dari implementasi; tidak ada basis data spesifik, penyedia cloud, atau bahasa pemrograman yang ditetapkan di sini. Pemilihan teknologi didelegasikan ke Layer L6 (Technology Profiles).

1. Logical Storage Domains (Domain Penyimpanan Logis)

Sistem penyimpanan dipecah secara logis berdasarkan karakteristik mutabilitas dan akses epistemologis (Custody).

- **Immutable Raw Data Lake:** Domain penyimpanan *append-only* untuk data deret waktu murni dan log kejadian (event logs). Setelah direkam dan di-hash (fingerprinted), data di sini tidak akan pernah bisa dimutasi.
- **Processed Feature Fabric:** Domain penyimpanan untuk *Feature Vectors* (S5) yang telah dikomputasi. Domain ini harus memisahkan secara fisik blok data *In-Sample* (IS) dan *Out-of-Sample* (OOS).
- **Relational/Graph Ontology Store:** Domain penyimpanan transaksional yang mengelola status entitas, *Hypothesis Lifecycle*, *Lineage Edges*, dan *Custody Receipts*. Domain ini harus mendukung jaminan ACID penuh untuk mencegah *race condition* saat persetujuan tata kelola (governance gates).
- **Artifact Blob Vault:** Domain penyimpanan objek hierarkis untuk entitas berukuran besar (misalnya: file model biner, *Validation Reports* PDF, *Knowledge Objects* yang diarsipkan). Entitas dalam vault ini diindeks murni melalui hash kriptografinya.

2. Logical Compute Domains (Domain Komputasi Logis)

Komputasi dipisahkan menjadi beberapa zona eksekusi untuk memastikan determinisme dan melindungi integritas *out-of-sample*.

- **Research Sandbox Domain:** Zona komputasi baca-saja yang digunakan oleh peneliti untuk eksplorasi *In-Sample* (Discovery). Memiliki akses tak terbatas ke Data Lake partisi historis, namun akses keluarnya (egress) diblokir untuk mencegah eksfiltrasi data.
- **Execution & Orchestration Domain:** Lapisan kontrol (*Control Plane*) yang mengatur siklus hidup komputasi, menjadwalkan *Experiment Runs*, menginjeksi *deterministic seeds*,

dan memonitor eksekusi *Directed Acyclic Graph* (DAG). Domain ini mendelegasikan beban kerja secara paralel tetapi mempertahankan status eksekusi utama.

- **Validation & Simulation Engine:** Zona komputasi terisolasi dengan akses eksklusif untuk menjalankan simulasi mikrostuktur dan *Combinatorially Symmetric Cross-Validation* (CSCV). Node komputasi dalam domain ini bersifat *ephemeral* (sementara) dan *stateless*; mereka dihancurkan segera setelah *Validation Report* dihasilkan.

3. Artifact Architecture (Arsitektur Artefak)

Setiap luaran komputasi (Feature, Experiment, Validation Report) tidak pernah disimpan sebagai file tunggal, melainkan sebagai sebuah **Artifact Bundle** standar.

Setiap *Artifact Bundle* minimal terdiri dari:

1. **Binary Payload:** Hasil mentah dari komputasi (vektor performa, model tersimpan).
2. **Metadata Manifest:** File deskriptor logis yang memetakan luaran ke Ontologi L3 (ID, Deskripsi).
3. **Lineage Envelope:** Jejak kriptografis tak terputus yang memuat (`run_id`, `dataset_fingerprint`, `code_commit`).
4. **Cryptographic Signature:** Hash SHA/BLAKE dari ketiga komponen di atas, digunakan sebagai *Primary Key* permanen artefak tersebut di dalam *Artifact Blob Vault*.

4. Repository Architecture (Arsitektur Repositori)

Kode sistem dan definisi riset diatur dalam topologi repositori yang mencegah pencampuran antara logika infrastruktur dan logika ilmiah.

- **Core OS Repository:** Berisi mesin L4 (Orchestrator, Statistical Engines, Custody Manager). Perubahan di sini mengubah hukum fisika dari platform riset dan membutuhkan persetujuan tata kelola tertinggi (Orang/Tim Platform).
- **Feature Library Repository:** Kumpulan definisi matematis *Feature* (DAG Nodes). Harus di-versi secara ketat. Tidak boleh ada mutasi kode historis; perbaikan bug pada sebuah *feature* membutuhkan *Feature ID* baru.
- **Strategy & Hypothesis Repository:** Tempat di mana peneliti mendefinisikan *Economic Mechanisms*, *Hypotheses*, dan konfigurasi *Experiment*. Repositori ini berjalan di atas *Feature Library* dan memanfaatkan *Core OS* untuk eksekusi.

5. Security Boundaries (Batas Keamanan & Custody)

Keamanan dalam Research OS bukan sekadar keamanan siber, melainkan **Keamanan Epistemologis** (melindungi validitas ilmiah).

- **The Data Fence (Batas In-Sample vs OOS):** Pemisahan fisik dan kredensial akses antara data yang terlihat oleh peneliti (IS) dan data yang ditahan oleh sistem untuk pengujian (OOS).
- **Machine-to-Machine (M2M) Identity:** *Execution Engine* yang memproses data OOS beroperasi menggunakan *Service Principal* terisolasi yang kredensialnya tidak dapat

diakses oleh peneliti manusia mana pun.

- **Custody Token Interceptors:** Semua antarmuka baca (Read APIs) menuju *Raw Data Lake* atau *Processed Fabric* dibungkus oleh sebuah interseptor proksi. Proksi ini akan menolak akses yang tidak menyertakan *Custody Receipt* valid yang diotorisasi oleh *Custody Manager*.

6. Execution Environments (Lingkungan Eksekusi)

Infrastruktur harus mendukung dua bayangan cermin (*mirrored*) dari lingkungan eksekusi untuk menjamin portabilitas absolut:

- **Local/Desk Environment:** Instansiasi ringan dari Runtime L4 yang berjalan di mesin peneliti. Lingkungan ini hanya diberi hak akses baca ke data *In-Sample*. Hasil eksperimen di sini murni untuk keperluan "Draft" dan tidak diakui oleh *Governance Controller*.
- **Institutional Cluster Environment:** Lingkungan eksekusi terdistribusi *air-gapped*. Ini adalah satu-satunya lingkungan yang diizinkan menerbitkan *Validation Report* resmi dan mengeluarkan *Accepted Knowledge*. Determinisme memastikan bahwa kode yang sama yang berjalan di Lingkungan Lokal akan menghasilkan hash biner yang sama di Lingkungan Cluster (dengan asumsi parameter *In-Sample* yang identik).

7. Deployment Boundaries (Batas Penerapan)

- **Ingress Isolation:** Pipeline data masuk (feed handlers) diisolasi sepenuhnya dari node analitik. Data baru menulis ke lapisan *staging* sebelum digabungkan (*merged*) dan di-hash ulang sebagai *Epoch* baru di Data Lake.
- **Egress Air-Gap:** Zona komputasi riset tidak memiliki akses internet. Resolusi dependensi kode harus dilakukan melalui repositori artefak internal yang terkontrol.
- **Capital Boundary (Batas Kapital):** Pemisahan jaringan yang kaku antara Research OS dan Production/Trading OS. Artefak dari Research OS hanya dipromosikan melintasi batas ini melalui bus pesan asinkron searah (unidirectional message bus) setelah mencapai status *Accepted Knowledge*.

8. Implementation Conformance Rules (Aturan Kesesuaian Implementasi)

Siapa pun yang merancang implementasi fisik (L6/L7) di masa depan **HARUS** mematuhi aturan kesesuaian berikut:

1. **Immutability Mandate:** Basis data operasional yang dipilih untuk *Ontology Store* harus secara bawaan atau terkonfigurasi untuk menolak pernyataan UPDATE atau DELETE pada catatan *Lineage* dan *Failure Library*.
2. **Clock-Independence:** Sistem penjadwalan dan orkestrasi yang dipilih tidak boleh menggunakan *wall-clock time* mesin sebagai *seed* acak implisit dalam node komputasi mana pun.
3. **Namespace Partitioning:** Teknologi komputasi harus mendukung partisi *namespace* di tingkat memori/proses sehingga eksekusi konkuren dari eksperimen yang sama (tetapi dengan parameter yang berbeda) tidak dapat membocorkan (*leak*) *state* satu sama lain.

4. **Serialization Parity:** Standar serialisasi biner (seperti Parquet, Protobuf, dll.) yang dipilih harus menjamin urutan kolom dan tata letak memori (memory layout) yang deterministik agar *cryptographic hash* konsisten di semua lingkungan.

User prompt: You are the Chief Enterprise Architect of the Institutional Research Operating System. The current L5 Reference Architecture Specification is architecturally sound and structurally complete. Your task is NOT to redesign the architecture. Your task is to perform an institutional architecture refinement pass that ensures the document remains a pure Reference Architecture positioned strictly between: L4 Runtime Architecture ↓ L5 Reference Architecture ↓ L6 Technology Profiles The objective is to remove every implementation-specific leakage while strengthening the logical architecture. This is an architecture refinement only. Do NOT introduce new runtime components. Do NOT redesign the Runtime. Do NOT change the Ontology. Do NOT change Governance. Do NOT change Lineage. Instead, improve abstraction quality.

OBJECTIVES 1. Review the entire document for implementation leakage. Replace every implementation-specific concept with a logical architectural equivalent. Examples include (but are not limited to): ACID SHA BLAKE PDF Parquet Protobuf Message Bus REST API SQL Filesystem Object Storage Cloud Container Cluster Thread Memory Layout Database Programming Language Scheduling Framework Cryptographic Algorithm Names Serialization Technology Vendor terminology Do not merely delete them. Replace them with implementation-independent architectural concepts.

2. Strengthen the distinction between: Logical Domains Logical Boundaries Logical Responsibilities Logical Interactions Logical Contracts Logical Isolation Logical Ownership Avoid describing physical realization.

3. Replace storage-centric descriptions with knowledge-centric architecture. Prefer concepts such as: Knowledge Acquisition Domain Knowledge Processing Domain Knowledge Validation Domain Knowledge Governance Domain Knowledge Publication Domain Knowledge Preservation Domain instead of implementation-oriented storage descriptions whenever appropriate.

4. Add a new canonical section: Reference Interaction Model Describe the logical interactions between all major domains. Include: Research Domain Knowledge Domain Validation Domain Governance Domain Publication Domain Capital Boundary Knowledge Archive For every interaction specify: Purpose Direction Ownership Required Contract Required Lineage Required Governance Failure Behaviour No technology. Only architecture.

5. Add a new canonical section: Reference Architecture Principles These principles shall govern every future implementation. Include principles such as: Separation of Knowledge and Infrastructure Deterministic Promotion Immutable Knowledge Explicit Contracts Domain Isolation Technology Independence Scientific Integrity Lineage Preservation Governance First Implementation Replaceability These principles shall become the architectural laws of L5.

6. Strengthen the Conformance Rules. Conformance rules must describe architectural obligations rather than technology constraints. Examples: Every implementation shall preserve immutable lineage. Every implementation shall preserve deterministic execution. Every implementation shall preserve custody boundaries. Every implementation shall preserve governance semantics. Every implementation shall preserve reproducibility. No implementation may alter ontology semantics. No implementation may bypass architectural contracts.

7. Review every paragraph. Whenever a paragraph answers: "How is this implemented?" rewrite it so that it answers: "What architectural responsibility exists?" The entire document must remain implementation-independent.

8. Produce an Architecture Review Appendix. The appendix shall include: Architectural decisions retained Architectural decisions refined Implementation leakages removed Abstraction improvements Layer boundary verification L4/L5 separation verification L5/L6 separation verification No implementation choices shall remain.

Constraints Do NOT reduce technical rigor. Do NOT shorten the document. Do NOT simplify concepts. Increase abstraction quality while preserving precision. The final document shall represent a canonical institutional Reference Architecture that can support multiple future Technology Profiles without modification.

Response: L5 Reference Architecture Specification — Institutional Research OS

Layer: L5 — Reference Architecture

Status: Canonical Logical Specification

Owner: Chief Enterprise Architect

This specification defines the architectural topology, logical domains, interaction contracts, and isolation boundaries of the Institutional Research Operating System. As the canonical L5 Reference Architecture, this document acts as the rigid structural membrane between the behavioral semantics of the Runtime Architecture (L4) and the physical realization of the Technology Profiles (L6).

This document describes purely **what** architectural responsibilities exist and how they are topologically isolated. It completely abstracts **how** these responsibilities are physically implemented. No implementation may bypass the logical contracts defined herein.

1. Reference Architecture Principles

These principles serve as the immutable architectural laws of the L5 layer. Every future physical implementation (L6/L7) must mathematically and structurally guarantee these principles.

1. **Separation of Knowledge and Infrastructure:** The logical representation of scientific knowledge must be completely decoupled from the transient infrastructure that computes or stores it.
2. **Deterministic Promotion:** Artifacts move across logical domains exclusively via deterministic, contract-driven promotion. Manual or out-of-band state mutation is structurally prohibited.
3. **Immutable Knowledge:** Once a knowledge artifact achieves a terminal logical state, its representation is strictly append-only.
4. **Explicit Contracts:** All inter-domain communication occurs through explicit, versioned logical contracts. There are no implicit dependencies or hidden data flows.
5. **Domain Isolation:** Execution contexts and knowledge boundaries are structurally partitioned. Failure or compromise in one domain must be mathematically contained within its defined boundary.
6. **Technology Independence:** The validity of a scientific artifact must be entirely agnostic to the underlying hardware, scheduling mechanisms, or storage mediums.
7. **Scientific Integrity:** Epistemological security (Custody) takes absolute precedence over system performance, availability, or operational convenience.

8. **Lineage Preservation:** Every derived state must possess an unbroken, verifiable causal chain back to its axiomatic origin.
9. **Governance First:** Governance is not a systemic overlay; it is the fundamental structural gatekeeper. No logic executes, and no state transitions, without prior governance authorization.
10. **Implementation Replaceability:** Any L6 technology stack fulfilling an L5 domain contract can be entirely swapped without altering the mathematical validity of the Research OS.

2. Logical Knowledge Domains

The reference architecture discards medium-specific storage models in favor of Knowledge Domains. These domains separate information based on its epistemological maturity, mutability, and custody state.

- **Knowledge Acquisition Domain:** The foundational logical boundary for all external stimuli and raw empirical observations. It acts as an append-only, immutable temporal ledger. Once an observation enters this domain and is assigned a deterministic identity, its structural representation can never be altered.
- **Knowledge Processing Domain:** The domain responsible for housing derived structural data (e.g., resolved features). This domain requires rigorous logical partitioning, enforcing a structural void between *In-Sample* exploratory state and *Out-of-Sample* verification state.
- **Knowledge Governance Domain:** The highly consistent, transactional domain governing the Institutional Ontology, Hypothesis Lifecycle, Lineage Edges, and Custody Receipts. This domain holds the absolute truth of system state and must mathematically guarantee the prevention of race conditions during governance transitions.
- **Knowledge Preservation Domain:** The terminal repository for finalized, structured scientific artifacts. Assets within this domain are decoupled from their relational metadata and are strictly addressable via their intrinsic logical identity (cryptographic signatures).

3. Logical Execution Domains

Execution topology is strictly partitioned to protect out-of-sample integrity and enforce deterministic behavior across all computational environments.

- **Discovery Execution Domain:** An exploratory computational boundary granted read access exclusively to *In-Sample* partitions within the Knowledge Domains. To prevent epistemological contamination, this domain is subjected to absolute egress isolation; no resultant state may implicitly exit this domain into the institutional canon.
- **Orchestration & Control Domain:** The centralized, determinism-enforcing control plane. It holds responsibility for translating defined research DAGs into isolated execution contexts, injecting deterministic parameters, maintaining operational causality, and recording the execution lineage.
- **Validation Execution Domain:** A highly restricted, ephemeral computational boundary. This domain possesses exclusive contractual authority to access *Out-of-Sample* partitions for structural simulation and falsification testing. Execution units within this domain must be entirely stateless, existing only for the exact duration of the evaluation contract, after which both the state and the execution unit are logically annihilated.

4. Logical Artifact Architecture

Outputs within the system are never modeled as arbitrary files or unstructured data. Every resultant scientific entity is structurally mandated to be a **Canonical Artifact Bundle**, which acts as the universal unit of knowledge transfer.

Every Artifact Bundle must natively encapsulate:

1. **Logical Payload:** The abstract, mathematically deterministic result of the execution.
2. **Ontological Manifest:** The formal semantic descriptors mapping the payload explicitly to the L3 Data Ontology.
3. **Lineage Envelope:** The unbroken causal graph detailing the precise inputs, execution context, and dependencies responsible for the payload.
4. **Identity Signature:** A deterministic, cryptographic identifier derived from the absolute entirety of the Payload, Manifest, and Lineage. This signature acts as the immutable primary key for the artifact across all domains.

5. Logical Instruction Repositories

The definitions of system behavior and scientific inquiry are isolated into distinct structural boundaries to prevent the conflation of infrastructure logic with scientific hypothesis.

- **Platform Instruction Domain:** The logical repository containing the core engine abstractions, deterministic rulesets, and governance enforcement mechanisms. Changes here alter the fundamental "laws of physics" of the research platform and require supreme institutional authorization.
- **Structural Instruction Domain:** The strictly versioned repository holding the mathematical abstractions of the research pipeline (e.g., Feature definitions). This domain is strictly append-only; historical definitions are logically frozen, and refinements demand entirely new identities to preserve reproducibility.
- **Theoretical Instruction Domain:** The operational space where researchers formally define Economic Mechanisms, Hypotheses, and Experimental parameters. This domain depends on the Structural Domain but is entirely subordinate to the Platform Domain's constraints.

6. Epistemological Boundaries

Security in this architecture is not defined by traditional access control, but by the structural preservation of scientific validity and custody.

- **The Custody Fence:** A structural and operational partition mandating that exploratory algorithms cannot mathematically reach reserved verification data.
- **Systemic Operational Identity:** Validation executions must operate under autonomous, non-human systemic identities. No human actor, regardless of organizational authority, possesses the logical capability to directly view or interact with the validation data plane.
- **Custody Policy Interceptors:** Every logical interaction attempting to read from a Knowledge Domain must traverse a contractual interceptor. The interceptor's sole

responsibility is to instantly deny access unless presented with a mathematically valid, unexpired Custody Receipt issued by the Knowledge Governance Domain.

7. Logical Deployment Boundaries

- **Ingress Isolation:** The boundary where external observations are synthesized. This pipeline is fully decoupled from the core research domains. Observations are staged, verified, and deterministically stamped before being formally admitted to the Knowledge Acquisition Domain.
- **Egress Air-Gap:** The computational domains are logically severed from external state changes. All dependency resolutions must be statically defined and routed through internally controlled, immutable proxy registries.
- **The Capital Boundary:** The ultimate, unidirectional logical gateway separating the Research Operating System from the institutional Production/Trading Operating System. Artifacts may only cross this boundary upon achieving the status of *Accepted Knowledge*, passing through a rigidly defined, asynchronous promotion contract.

8. Reference Interaction Model

This section defines the mandatory logical contracts between major architectural domains.

Interaction 1: Research Domain ↔ Knowledge Acquisition/Processing Domain

- **Purpose:** Epistemological exploration and discovery feature engineering.
- **Direction:** Read-only from Knowledge; strictly constrained Write-only to Discovery buffers.
- **Required Contract:** In-Sample Custody Receipt.
- **Required Lineage:** Read events mathematically logged to the user's execution identity.
- **Failure Behaviour:** Instant isolation of the execution thread; no data is returned.

Interaction 2: Orchestration Domain ↔ Validation Domain

- **Purpose:** Execution of the formal Validation Framework (CSCV, Falsification).
- **Direction:** Bidirectional (Orchestration injects context → Validation returns Terminal Payload).
- **Required Contract:** Ephemeral Systemic Identity Token; Valid Out-of-Sample Custody Receipt.
- **Required Lineage:** Full Context Graph injection; Deterministic Seed capture.
- **Failure Behaviour:** Payload annihilated; deterministic execution failure recorded to the Knowledge Governance Domain.

Interaction 3: Knowledge Governance Domain ↔ Validation Domain

- **Purpose:** Verification of custody and ontological state prior to execution.
- **Direction:** Query/Response.

- **Required Contract:** State query must match the L3 Ontology transition matrices.
- **Failure Behaviour:** Complete halt of the Validation execution; invalid transition attempted.

Interaction 4: Knowledge Governance Domain ↔ Capital Boundary

- **Purpose:** Promotion of Accepted Knowledge to live market deployment.
- **Direction:** Unidirectional (Push from Governance to Capital).
- **Required Contract:** Fully satisfied Peer Review Sign-Off; Complete Cryptographic Lineage Envelope.
- **Required Governance:** Final CRO institutional cryptographic signature.
- **Failure Behaviour:** Promotion is structurally blocked; alert issued to Governance oversight.

9. Implementation Conformance Rules

Any L6 or L7 physical implementation claiming conformity to the Institutional Research OS MUST mathematically and structurally obey the following architectural obligations:

1. **Lineage Immutability Obligation:** The implementation shall natively reject any operation (logical or physical) that attempts to mutate, truncate, or overwrite a finalized Lineage Envelope or Failure Library Entry.
2. **Execution Determinism Obligation:** The implementation shall guarantee that execution schedulers, resource allocators, and operational environments inject zero temporal, localized, or systemic variance into the computational payload.
3. **Custody Preservation Obligation:** The implementation shall structurally partition memory and processing contexts such that concurrent execution boundaries share zero observable state. Custody boundaries must be enforced at the lowest possible layer of the execution environment.
4. **Logical State Serialization Parity:** The implementation shall enforce a standardized, deterministic memory and payload layout to guarantee that identical scientific inputs yield exactly identical identity signatures, irrespective of the physical execution hardware.
5. **Ontological Inviolability:** No implementation may bypass, extend, or alter the semantic definitions dictated by the L3 Data Ontology. Data structures must remain subservient to the logical ontology.

10. Architecture Review Appendix

Architectural Status: VERIFIED

- **Architectural Decisions Retained:** The fundamental tripartite execution model (Discovery, Orchestration, Validation), the rigorous custody model, the strict separation of In-Sample/Out-of-Sample, and the unidirectional Capital Boundary.
- **Architectural Decisions Refined:** Storage has been strictly elevated to "Knowledge Domains." Execution environments are now purely "Logical Domains" rather than hardware/deployment locales.

- **Implementation Leakages Removed:** All references to Parquet, Protobuf, ACID, SQL, Cloud, Containerization, specific hashing algorithms (SHA/BLAKE), Blob vaults, physical message buses, and API paradigms (REST) have been annihilated.
 - **Abstraction Improvements:** Artifacts are now defined by their inherent cryptographic/logical identity rather than physical file properties. Security is framed entirely as Epistemological Boundary enforcement rather than network security.
 - **Layer Boundary Verification:** * *L4/L5 Separation:* L4 maintains the *laws of physics* (state machines, semantics). L5 maintains the *topology and contracts* (where logic lives and how boundaries interact).
 - *L5/L6 Separation:* L5 contains zero vendor, technology, or protocol specifications. The choice of how to physically build the Knowledge Governance Domain (e.g., using a specific RDBMS or Graph Database) is formally delegated to L6.
-